Encryption

Reliable encryption is an important part of any web app building framework. Laravel uses OpenSSL to provide AES-256 and AES-128 encryption. All of the encrypted values are signed with the MAC (message authentication code) so that they are unique and the underlying values cannot be modified once the encryption is done.

```
<?php
namespace App\Http\Controllers;
use App\User;
use Illuminate\Http\Request;
use App\Http\Controllers\Controller;
class UserController extends Controller
{
    /**
     * Store a secret message for the user.
     * @param  Request  $request
     * @param  int  $id
     * @return Response
     */
    public function storeSecret(Request $request, $id)
    {
        $user = User::findOrFail($id);
        $user->fill([
            'secret' => encrypt($request->secret)
        ])->save();    }}
```

The encryption values are passed through the 'serialize' method during encryption, which allows for the encryption of the various objects and arrays. The non PHP clients receiving these encrypted values would need the 'unserialize' method to decrypt the data and render it useful. A 'decrypt' helper can also be used. If the MAC is invalid while decryption, an exception error will pop up. If you want to do this without serialisation, the 'encryptString' and 'decryptString' methods can also be used as follows:

```
use Illuminate\Support\Facades\Crypt;
$encrypted = Crypt::encryptString('Hello world.');
$decrypted = Crypt::decryptString($encrypted);
```

To catch the exception in case of an invalid MAC address while decryption, the following logic can be implemented:

```
use Illuminate\Contracts\Encryption\DecryptException;
```